Tema finale ISS 2026
Sprint 0: analisi dei requisiti

Edoardo Buttazzi

1 Obiettivo dello Sprint 0

Il sistema da realizzare, chiamato `cargoservice`, ha il compito di automatizzare il caricamento dei container nella stiva di una nave, utilizzando un robot per spostarli: il `cargorobot`. Il servizio segue ogni operazione dall'accettazione della richiesta al deposito finale, passando per la prenotazione di uno slot e la marcatura del container.

Nello Sprint 0 vengono stabiliti il funzionamento del sistema e una prima bozza dell'architettura.

2 Il sistema e i suoi requisiti

La *hold* è uno spazio rettangolare suddiviso in quattro slot di stoccaggio, ciascuno destinato a un singolo container, più uno slot temporaneo (`slot5`) riservato all'operazione di marcatura. La `IOPort` comprende un pulsante per le richieste e un display; un sonar rileva la presenza dei container. Si indica con D la distanza istantanea misurata dal sensore e con $DFREE$ il valore di riferimento associato all'ostacolo fisso in assenza di carico.

La tabella riassume i requisiti funzionali e le relative ipotesi operative introdotte nel prototipo.

ID	Requisito
R1	**Gestione delle richieste.** Il servizio deve ricevere le richieste di caricamento inviate tramite il pulsante della IOPort. Deve rispondere `retrylater` se la porta è occupata, il sistema è fuori servizio oppure un caricamento è già attivo. Se tutti i quattro slot finali sono occupati, deve rifiutare la richiesta.
R2	**Accettazione e prenotazione.** Quando accetta una richiesta, il servizio deve prenotare lo slot finale libero con indice minore, comunicarne il nome al cliente ed entrare nello stato *engaged*. Lo slot passa da `FREE` a `RESERVED` e rimane associato all'operazione fino al deposito finale oppure al rilascio per timeout.
R3	**Attesa e rilevamento del container.** Dopo l'accettazione, il servizio deve attendere la presenza del container, che viene confermata quando $D < DFREE/2$ permane per 3 secondi continui. Se la conferma non arriva prima dei 30 secondi dall'accettazione, il servizio deve liberare la prenotazione e tornare *disengaged*, senza avviare il robot.
R4	**Rilevamento del fuori servizio.** Se il sonar misura $D > DFREE$ per almeno 3 secondi consecutivi, il servizio deve passare a `OUT_OF_SERVICE`. Finché il sistema è fuori servizio, le nuove richieste devono ricevere `retrylater`.

ID	Requisito
R5	**Trasporto e marcatura.** Dopo la conferma della presenza, il coordinatore deve richiedere il trasporto del container dalla IOPort a `slot5`, dove avviene la marcatura. Il trasporto verso lo slot prenotato deve iniziare soltanto dopo la segnalazione di completamento del marker. Alla conferma del deposito finale, lo slot passa a `OCCUPIED` e il servizio torna *disengaged*.
R6	**Segnalazione mediante LED.** Il LED deve lampeggiare per tutto il tempo in cui il sistema è *engaged* e deve essere spento quando il sistema è *disengaged*.

3 Modello del sistema

3.1 Ipotesi da validare

A partire dall'analisi e dalla sintesi dei requisiti di consegna, emergono alcuni aspetti del comportamento non completamente definiti. Per poter descrivere il ciclo di caricamento e formulare test precisi, si adottano le seguenti ipotesi, da validare con il committente.

A1 – Tempi di attesa. Si assume un tempo massimo di arrivo $T_a = 30\,\text{s}$ e un intervallo di conferma della presenza $T_p = 3\,\text{s}$. Il timeout parte dall'accettazione della richiesta e, in questa prima interpretazione, la presenza deve essere confermata prima della scadenza.

A2 – Una sola operazione alla volta. Il servizio gestisce un caricamento alla volta. Se riceve una nuova richiesta mentre è già *engaged*, risponde `retrylater` e prosegue con l'operazione in corso.

A3 – Termine dell'operazione. Se scade il timeout di attesa del container, lo slot prenotato torna libero. Se invece il caricamento termina correttamente, lo slot diventa occupato e il servizio torna *disengaged*.

A4 – Scelta dello slot e risposta di rifiuto. Si prenota lo slot libero con indice minore. Se la stiva è piena, si mostra un messaggio di rifiuto sul display.

A5 – IOPort e dispositivi fisici. Si prevede di rappresentare il pulsante e il display della IOPort attraverso un'interfaccia web: il cliente potrà inviare una richiesta di carico e visualizzarne la risposta, insieme allo stato della stiva e del servizio. Per il sonar e il LED si prevede invece l'impiego di una scheda Pico W, alla quale saranno collegati i due dispositivi.

3.2 Dalla richiesta al deposito finale

Alla ricezione di una richiesta, il servizio restituisce `retrylater` se la IOPort risulta occupata, se il sistema è fuori servizio oppure se è già in corso un caricamento. Se la stiva è piena, la richiesta viene rifiutata.

Negli altri casi viene scelto lo slot libero con indice minore, prenotato e mostrato il nome sul display. Il servizio memorizza lo slot assegnato all'operazione in corso e ne cambia lo stato da `FREE` a `RESERVED`. Il sistema entra quindi in `WAIT_CONTAINER` e avvia il timeout.

Se il container non viene confermato entro il tempo disponibile, il servizio libera la prenotazione e torna in attesa di nuove richieste. Se invece la presenza viene confermata in tempo e il servizio è operativo, interrompe il timeout e chiede al robot di portare il container in `slot5`.

Per il movimento verso la cella di accesso a `slot5`, `cargoservice` invia a `cargorobot` una richiesta con la destinazione da raggiungere e ne attende l'esito. In caso di successo, il servizio aggiorna lo stato dello spostamento ad `ARRIVED`, senza liberare lo slot né concludere il caricamento. Se il robot segnala un errore, il servizio registra `FAILED` e passa fuori servizio.

La conferma del deposito finale conclude l'operazione: lo slot diventa `OCCUPIED`, l'assegnazione viene cancellata e il servizio torna *disengaged*, spegnendo il LED. Il display viene aggiornato per mostrare il nuovo stato della stiva.

```
DISENGAGED -- richiesta accettata --> WAIT_CONTAINER
WAIT_CONTAINER -- timeout --> DISENGAGED
WAIT_CONTAINER -- presenza confermata --> TO_SLOT5
TO_SLOT5 -- deposito temporaneo confermato --> WAIT_MARKER
WAIT_MARKER -- marcatura completata --> TO_FINAL
TO_FINAL -- deposito finale confermato --> DISENGAGED
```

3.3 Fasi del caricamento

Per descrivere il comportamento conviene distinguere due informazioni: la fase raggiunta dal caricamento e la disponibilità del servizio. La prima indica che cosa sta facendo il sistema, la seconda segnala se il servizio funziona regolarmente o se è stata rilevata un'anomalia. Nel modello si usano i seguenti nomi:

```
phase = DISENGAGED | WAIT_CONTAINER | TO_SLOT5
        | WAIT_MARKER | TO_FINAL
serviceStatus = WORKING | OUT_OF_SERVICE
navigationState = IDLE | MOVING | ARRIVED | FAILED
```

`DISENGAGED` indica l'assenza di un caricamento attivo. Le altre fasi descrivono, nell'ordine, l'attesa del container, il trasporto verso `slot5`, l'attesa della marcatura e il trasporto finale. Durante queste quattro fasi il sistema è *engaged* e il LED lampeggia.

Lo stato della navigazione distingue l'assenza di un comando, il movimento in corso, l'arrivo e il fallimento ed è gestito dall'attore `cargorobot`, che ne comunica l'esito a `cargoservice`. L'arrivo alla cella di accesso a `slot5` non conferma il deposito temporaneo (WAIT_MARKER), ma resta `TO_SLOT5` fino alla conferma dal deposito.

La disponibilità (serviceStatus) viene mantenuta separata perché il sonar potrebbe segnalare un'anomalia anche durante un caricamento. In quel caso il servizio passa a `OUT_OF_SERVICE` e rifiuta nuove richieste.

3.4 Layout e gestione operativa

A livello di modello del dominio, la rappresentazione dello stato e dello spazio di lavoro è suddivisa in quattro strutture logiche distinte, ciascuna responsabile di un aspetto specifico:

Layout. Contiene la mappa, le aree degli slot e i punti di accesso del robot.

HoldInventory. Gestisce gli stati FREE, RESERVED e OCCUPIED degli slot 1–4. Serve a scegliere e prenotare uno slot senza occuparsi delle coordinate.

MarkingBuffer. Traccia l'eventuale container presente nell'area temporanea di marcatura (`slot5`), mantenendola separata dai quattro slot di destinazione finale.

LoadOperation. Contiene i dati dell'operazione di carico in corso: identificatore, slot assegnato, fase attuale, scadenze e spostamento attivo.

Ogni slot è associato alla propria posizione nel layout tramite un identificatore. La gestione delle prenotazioni rimane così separata dalla rappresentazione geometrica della stiva.

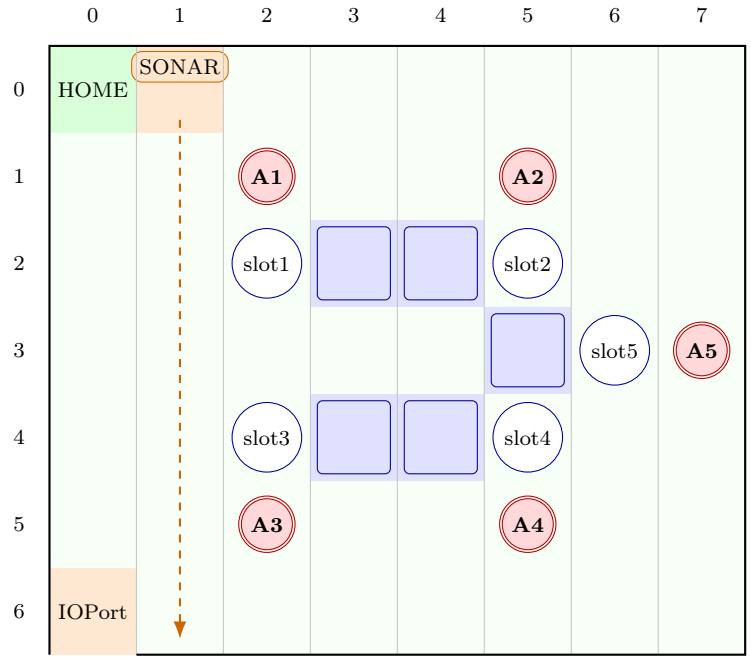
I quattro slot finali possono essere `FREE`, `RESERVED` oppure `OCCUPIED`. Uno slot prenotato non è disponibile per altre richieste, anche se non contiene ancora il container, e diventa occupato alla conferma del deposito.

`slot5` è riservato alla marcatura e non viene considerato nella ricerca di uno slot disponibile. La sua occupazione temporanea viene registrata separatamente nel buffer specifico.

3.4.1 Configurazione spaziale

Per descrivere la disposizione della stiva e individuare le destinazioni del robot, si propone una rappresentazione bidimensionale a griglia, composta da sette righe e otto colonne. Ogni cella è identificata da una coppia di indici `cells[row][column]`.

La mappa distingue la posizione iniziale del robot (`HOME`), la `IOPort`, gli slot finali `slot1`–`slot4` e l'area temporanea di marcatura `slot5`. I punti indicati con A_n, rappresentano le celle di accesso preferenziale ai rispettivi slot.



La mappa di navigazione del progetto definisce gli spazi effettivamente percorribili. Il modulo Layout ne offre una vista in lettura, aggiungendo i relativi punti di accesso, evitando così una matrice modificabile di celle all'interno dell'Inventory.

A livello pratico, la logica di carico lavora unicamente con i nomi delle destinazioni. `cargoservice` invia il target logico a `cargorobot`, che si occupa di raggiungere il punto di accesso corrispondente nel `Layout`.

3.5 Tipi e vincoli del modello

I contratti seguenti illustrano la separazione proposta.

```
enum SlotState { FREE, RESERVED, OCCUPIED }
enum Target { HOME, IOPORT, SLOT1, SLOT2, SLOT3, SLOT4, SLOT5 }

interface Position {
    int row();
    int column();
}
interface Destination {
    Set<Position> area();
    Position accessPoint();
}
interface Layout {
    int rows();
    int columns();
    boolean isTraversable(Position position);
    Destination destination(Target target);
}

interface HoldInventory {
    SlotState stateOf(int slotId);
    OptionalInt reserveFirstAvailable();
    void confirmDeposit(int slotId, String containerId);
}
interface MarkingBuffer {
    Optional<String> containerId();
    void confirmDeposit(String containerId);
    String confirmCollection();
}
```

4 Prima architettura logica

4.1 Componenti forniti e componenti da sviluppare

Il sistema combina componenti software messi a disposizione dal committente con componenti da sviluppare per il servizio di caricamento. Tra le opzioni messe a disposizione dal committente figurano l'oggetto Java `RobotObj26` e i servizi `RobotService26` e `RobotSmart26`. Si sceglie `RobotSmart26` perché permette di indicare le coordinate di destinazione e delegare il calcolo del percorso. Questa soluzione riduce la distanza tra le primitive del robot e le operazioni richieste dall'applicazione, consentendo di concentrare lo sviluppo sulla gestione del caricamento.

Devono invece essere sviluppati quattro componenti: **cargoservice**, che gestisce le richieste e coordina il ciclo di caricamento. **cargorobot**, che traduce le destinazioni logiche nelle coordinate richieste da RobotSmart26 e comunica l'esito degli spostamenti. **ioportservice**, che collega l'interfaccia del cliente al coordinatore. **sensorservice**, che interpreta le misure del sonar e verifica la persistenza delle condizioni di presenza e anomalia.

Sono inoltre da sviluppare l'interfaccia web della IOPort, i modelli interni dei dati e il software di collegamento ai dispositivi sulla Pico W. Sonar, LED e marker sono dispositivi esterni con cui il software deve interagire. In particolare, le modalità di integrazione del marker e della sua segnalazione di completamento rimangono da chiarire con il committente.

4.2 Organizzazione dei componenti

L'architettura distingue i componenti attivi del sistema dagli oggetti che rappresentano i dati del dominio. I quattro servizi `cargoservice`, `ioportservice`, `cargorobot` e `sensorservice` sono modellati come attori QAK: ciascuno gestisce il proprio stato e reagisce ai messaggi ricevuti. La loro collaborazione avviene attraverso messaggi espliciti.
Si sceglie QAK perché consente di descrivere direttamente il comportamento dei servizi mediante stati, transizioni e scambi di messaggi. Il modello rende così espliciti il coordinamento del caricamento e le interazioni tra richiesta del cliente, rilevazioni del sensore e movimento del robot.

`Layout`, `HoldInventory`, `MarkingBuffer` e `LoadOperation` sono invece rappresentati mediante tipi Java. Non richiedono un comportamento autonomo né la ricezione di messaggi: descrivono dati e vincoli del dominio e vengono utilizzati attraverso chiamate ai metodi. cargoservice gestisce l'inventario, il buffer di marcatura e l'operazione in corso; cargorobot consulta il layout per la navigazione.

Componente	**Tipo**	**Responsabilità**
`cargoservice`	Attore QAK	Riceve le richieste, gestisce le prenotazioni e i tempi di attesa e coordina il caricamento. È il solo componente che modifica l'inventario, il buffer di marcatura e i dati dell'operazione in corso.
`cargorobot`	Attore QAK	Riceve target logici da `cargoservice`, gestisce lo stato della navigazione e comunica al coordinatore l'arrivo o il fallimento dello spostamento.
`ioportservice`	Attore QAK	Inoltra le richieste generate dal pulsante, aggiorna il display della GUI e fa da tramite per i comandi di stato del LED destinati al Pico W, in base alle direttive ricevute da cargoservice. `cargoservice`.
`sensorservice`	Attore QAK	Legge il sonar, verifica la persistenza temporale delle condizioni rilevate e comunica lo stato della porta e le eventuali anomalie.
Modelli interni	POJO	In `cargoservice`, `HoldInventory`, `MarkingBuffer` e `LoadOperation` conservano i dati del carico. In `cargorobot`, `Layout` descrive l'ambiente ed è consultato in sola lettura.
`RobotSmart26`	Servizio esterno	Pianifica ed esegue i movimenti del robot fisico e comunica l'esito delle richieste di navigazione a `cargorobot`.
`Software Pico W`	Firmware di gestione dei dispositivi.	Acquisisce e trasmette le misure del sonar; riceve i comandi per il LED e ne gestisce l'accensione, lo spegnimento e il lampeggio.

La IOPort costituisce il punto di interazione con l'utente e raggruppa logicamente i vari dispositivi. Il pulsante e il display sono gestiti tramite un'applicazione web, mentre il LED risiede fisicamente sulla scheda Pico W insieme al sonar. `ioportservice` funge da punto di raccordo per tutti questi elementi: riceve gli input dal cliente e inoltra le richieste a cargoservice, aggiorna la visualizzazione dello stato sul display e traduce le decisioni del coordinatore in comandi verso il LED.
La decisione su quando attivare o interrompere il lampeggio spetta a cargoservice in base alla fase di carico, notificando tale indicazione a ioportservice, che a sua volta istruisce il software sul Pico W incaricato del controllo del componente.

Per quanto riguarda il flusso in ingresso invece, il software sul Pico W campiona periodi-

camente le distanze del sonar e le trasmette a sensorservice, il quale ne valuta la persistenza temporale per rilevare anomalie o la presenza del carico prima di notificarle al coordinatore.

4.3 Attori, contesti e QAK

I quattro servizi sono rappresentati come QActor in contesti distinti. Gli estratti seguenti mostrano lo scheletro eseguibile dello Sprint0 e una prima definizione dei messaggi tra i componenti. Il ciclo completo di caricamento descritto nell'analisi sarà sviluppato negli sprint successivi.

`ctxcargoservice`**: Riceve le richieste di caricamento.** Se il servizio è impegnato, la porta è occupata o il sistema è fuori servizio, invita a riprovare. Se la stiva è piena, rifiuta; altrimenti prenota uno slot, ne comunica il nome e si considera impegnato.

```
Context ctxcargoservice ip [host="localhost" port=8020]
Request load_request : load_request(none)
Reply load_accepted : load_accepted(SLOT) for load_request
Reply retrylater : retrylater(none) for load_request
Reply load_refused : load_refused(none) for load_request

QActor cargoservice context ctxcargoservice {
    [#
        val inventory = HoldInventory()
        val serviceStatus = ServiceStatus.WORKING
        val ioportOccupied = false
        var engaged = false
    #]

    State waiting initial { }
    Transition t0 whenRequest load_request -> evaluate_request

    State evaluate_request {
        if [# engaged || ioportOccupied ||
              serviceStatus == ServiceStatus.OUT_OF_SERVICE #] {
            replyTo load_request with retrylater : retrylater(none)
        }
        else {
            [# val FreeSlot = inventory.reserveFirstAvailable() #]
            if [# FreeSlot.isEmpty #] {
                replyTo load_request with load_refused :
                    load_refused(none)
            }
            else {
                [#
                    val Slot = "slot${FreeSlot.asInt}"
                    engaged = true
                #]
                replyTo load_request with load_accepted :
                    load_accepted($Slot)
            }
        }
    }
    Goto waiting
}
```

`ctxioport`**: pulsante, risposte, display e LED.** Rileva la pressione del pulsante e inoltra la richiesta al servizio, mostrando successivamente nella console la risposta ricevuta.

```
Context ctxioport ip [host="localhost" port=8021]
```

```
Dispatch button_pressed : button_pressed(none)
Dispatch display_update : display_update(DATA)
Dispatch led_command : led_command(MODE)

QActor ioportservice context ctxioport {
    State waiting initial { }
    Transition t0
        whenMsg button_pressed -> send_request
        whenReply load_accepted -> show_response
        whenReply retrylater -> show_response
        whenReply load_refused -> show_response
        whenMsg display_update -> update_display
        whenMsg led_command -> update_led

    State send_request {
        request cargoservice -m load_request : load_request(none)
    }
    Goto waiting

    State show_response {
        onMsg(load_accepted : load_accepted(Slot)) {
            [# val Slot = payloadArg(0) #]
            println("[IOPORT] slot assegnato $Slot")
        }
        onMsg(retrylater : retrylater(none)) {
            println("[IOPORT] riprovare piu tardi")
        }
        onMsg(load_refused : load_refused(none)) {
            println("[IOPORT] stiva piena")
        }
    }
    Goto waiting
}
```

`ctxrobot`: **richiesta di spostamento.** Riceve una destinazione, conferma il movimento in corso e l'arrivo.

```
Context ctxrobot ip [host="localhost" port=8022]
Request reach_target : reach_target(TARGET)
Reply target_reached : target_reached(TARGET) for reach_target

QActor cargorobot context ctxrobot {
    State waiting initial { }
    Transition t0 whenRequest reach_target -> reaching

    State reaching {
        onMsg(reach_target : reach_target(Target)) {
            [# val Destination = payloadArg(0) #]
            println("[CARGOROBOT] spostamento verso $Destination")
            replyTo reach_target with target_reached :
                target_reached($Destination)
        }
    }
    Goto waiting
}
```

`ctxsensor`: **classificazione dei feedback del sonar.** Riceve una distanza e distingue possibile presenza del container, possibile anomalia e misura intermedia.

```
Context ctxsensor ip [host="localhost" port=8023]
Dispatch sonar_sample : sonar_sample(D)
Dispatch sensor_state : sensor_state(CONDITION)

QActor sensorservice context ctxsensor {
    [# val DFREE = 100.0 #]

    State waiting initial {}
    Transition t0 whenMsg sonar_sample -> evaluate_sample

    State evaluate_sample {
        onMsg(sonar_sample : sonar_sample(D)) {
            [# val Distance = payloadArg(0).toDouble() #]
            if [# Distance < DFREE / 2 #] {
                println("[SENSOR] presenza regolare: da confermare per 3 secondi di fila")
            }
            else {
                if [# Distance > DFREE #] {
                    println("[SENSOR] possibile anomalia: da confermare per 3 secondi di fila")
                }
                else {
                    println("[SENSOR] nessuna anomalia o presenza rilevata")
                }
            }
        }
    }
    Goto waiting
}
```

4.4 Comunicazione

Gli attori comunicano attraverso messaggi, mantenendo separate le rispettive responsabilità nei quattro contesti computazionali.

`ioportservice` inoltra a `cargoservice` le richieste di caricamento generate dal cliente e riceve una risposta di accettazione, rinvio o rifiuto. In caso di accettazione, la risposta indica anche lo slot prenotato.

`sensorservice` comunica al coordinatore la presenza del container e le eventuali anomalie rilevate dal sonar. `cargoservice` invia a `ioportservice` le informazioni necessarie per aggiornare il display e controllare il LED.

Per la movimentazione, `cargoservice` indica a `cargorobot` la destinazione da raggiungere e riceve l'esito dell'operazione. Durante l'attesa, il coordinatore continua a gestire le richieste del cliente e le segnalazioni del sensore.

4.5 Piani di test

T1 – Accettazione della richiesta Si imposta `slot1` come occupato, lasciando liberi gli altri slot, e si preme il pulsante. Il servizio deve prenotare `slot2`, mostrarne il nome sul display e far lampeggiare il LED. Il robot deve restare fermo in attesa del container.

T2 – Richiesta quando il servizio è già in uso Si provano separatamente due casi: una richiesta con la IOPort occupata e una nuova richiesta mentre il servizio attende un container. In entrambi i casi deve comparire `retrylater`, senza prenotare altri slot né avviare il robot.

T3 – Stiva piena Con IOPort libera e tutti i quattro slot occupati, si invia una richiesta. Il display deve mostrare il rifiuto e gli slot devono rimanere invariati.

T4 – Fuori servizio Si imposta $D = 110$ per almeno 3 secondi continui. Il display deve mostrare `Out of service`. Una successiva richiesta deve ricevere `retrylater`, senza creare prenotazioni.

T5 – Container non arrivato Si accetta una richiesta e si lascia la IOPort libera per 30 secondi. Il servizio deve liberare lo slot prenotato, spegnere il LED e aggiornare il display, mentre il robot deve rimanere fermo.

T6 – Movimento verso slot5 Dopo l'accettazione, si imposta $D = 40$ per almeno 3 secondi continui, completando l'intervallo prima dei 30 secondi di attesa. `cargoservice` deve richiedere a `cargorobot` il movimento verso la cella di accesso a `slot5` soltanto dopo la conferma della presenza. L'esito positivo deve aggiornare lo stato dello spostamento ad `ARRIVED`, lasciando lo slot finale `RESERVED`, la fase `TO_SLOT5` e il LED lampeggiante.

T7 – Container arrivato Dopo l'accettazione e il rilevamento del container, il robot deve trasportarlo e depositarlo in `slot5`. Il trasporto verso lo slot prenotato deve iniziare solo dopo il completamento della marcatura. Alla conferma del deposito finale, lo slot deve risultare occupato, `slot5` libero e il display aggiornato. Il LED deve lampeggiare durante l'operazione e spegnersi alla conclusione.

5 Obiettivo dello Sprint 1

Lo Sprint 1 implementerà l'intero flusso iniziale del sistema: ricezione della richiesta, prenotazione dello slot, attesa del container e navigazione del robot fino alla cella di accesso a `slot5`, con relativa verifica dell'esito.

A livello architetturale, il prototipo manterrà disaccoppiati il `Layout` e la gestione dell'`HoldInventory`, distribuendo i quattro QActor `cargoservice`, `ioportservice`, `cargorobot` e `sensorservice` nei contesti `ctxcargoservice`, `ctxioport`, `ctxrobot` e `ctxsensor`.

Nello Sprint 1 sonar e LED saranno simulati, mantenendo verificabili il rilevamento del container, la segnalazione di fuori servizio e il lampeggio durante il caricamento. Il collegamento ai dispositivi reali tramite Pico W verrà affrontato in uno sprint successivo.